

Qonaqzhai — Diploma Defense

Diploma Defense

Bahaidahar

May 2026

- Slide 1 — Title
- Qonaqzhai
- Slide 2 — Problem Statement
- Slide 3 — Solution at a Glance
- Slide 4 — Domain Model
- Slide 5 — Stack Overview
- Slide 6 — Backend Architecture
- Slide 7 — Service-to-Service Communication
- Slide 8 — Clean Architecture per Service
- Slide 9 — Web Frontend (Feature-Sliced Design)
- Slide 10 — Mobile Architecture
- Slide 11 — Data Model
- Slide 12 — Real-time Chat (WebSocket)
- Slide 13 — Stack Justification (Comparative)
- Slide 14 — Comparative Analysis (Competitors)
- Slide 15 — AI Event Planner (Signature Feature)
- Slide 16 — Security Model
- Slide 17 — Testing & Quality
- Slide 18 — Agentic Engineering & MCP
- Slide 19 — Live Demo Script (8 min)
- Slide 20 — Roadmap
- Slide 21 — Lessons Learned
- Slide 22 — Q&A
- Thank you.
- Appendix A — Numbers at a Glance
- Appendix B — How to Read This Repository

Slide 1 — Title

Qonaqzhai

AI-Powered Event Services Marketplace for Kazakhstan

Diploma Defense — May 2026

A bilingual (RU / KZ / EN) mobile-first marketplace that connects customers planning weddings, corporate events, and traditional Kazakh ceremonies with local vendors — venues, catering, photo/video, decor, music, traditional services. An integrated Claude-powered AI assistant guides the customer from idea to booked, paid event.

Slide 2 — Problem Statement

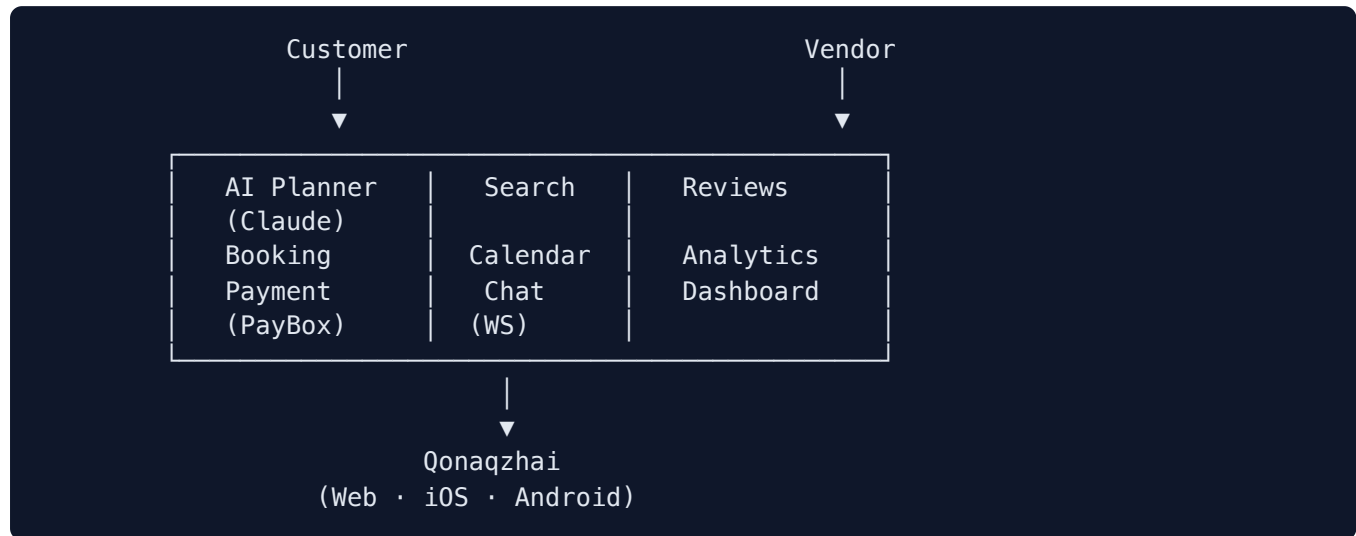
Why Kazakhstan Needs Qonaqzhai

Pain Point	Status Quo in KZ
Discovery	Instagram DMs, WhatsApp chains, word-of-mouth
Trust	No verified reviews; deposits paid into private cards
Language	No platform supports Kazakh + Russian + English equally
Tradition	Generic global platforms ignore тұсаукесер, беташар, шашу
Planning	Customer has to coordinate 5–10 vendors manually
Payment	No escrow; vendor takes deposit and disappears risk

Market gap: KZ wedding & events sector ≈ \$1.5B/year, fragmented across Instagram, Telegram, fl.kz, weddingsalon.kz, none of which solve the full journey.

Slide 3 — Solution at a Glance

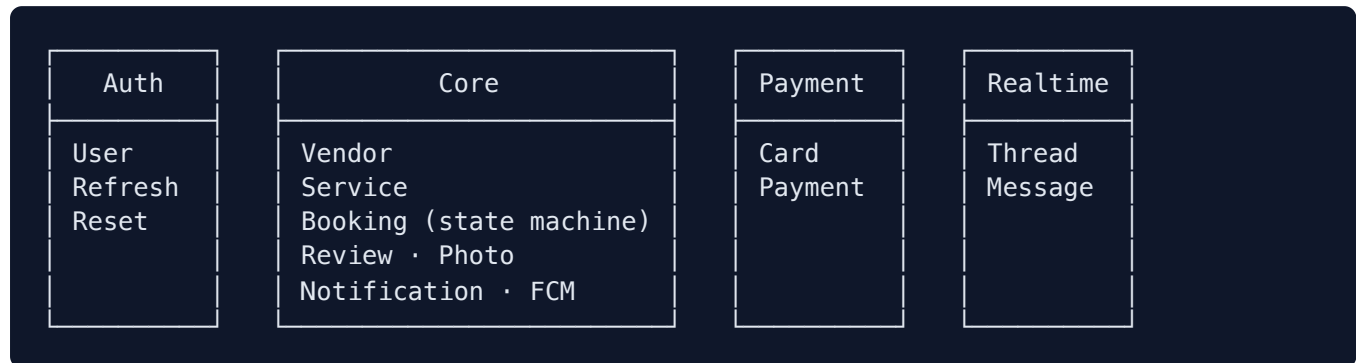
Qonaqzhai in One Diagram



Three interfaces, one platform: Next.js 16 web, Flutter mobile, Go microservices backend.

Slide 4 — Domain Model

Bounded Contexts



Booking lifecycle: pending → accepted → completed → paid (with branches declined , cancelled). Implemented as an explicit state machine in core/usecase/booking/ .

Slide 5 — Stack Overview

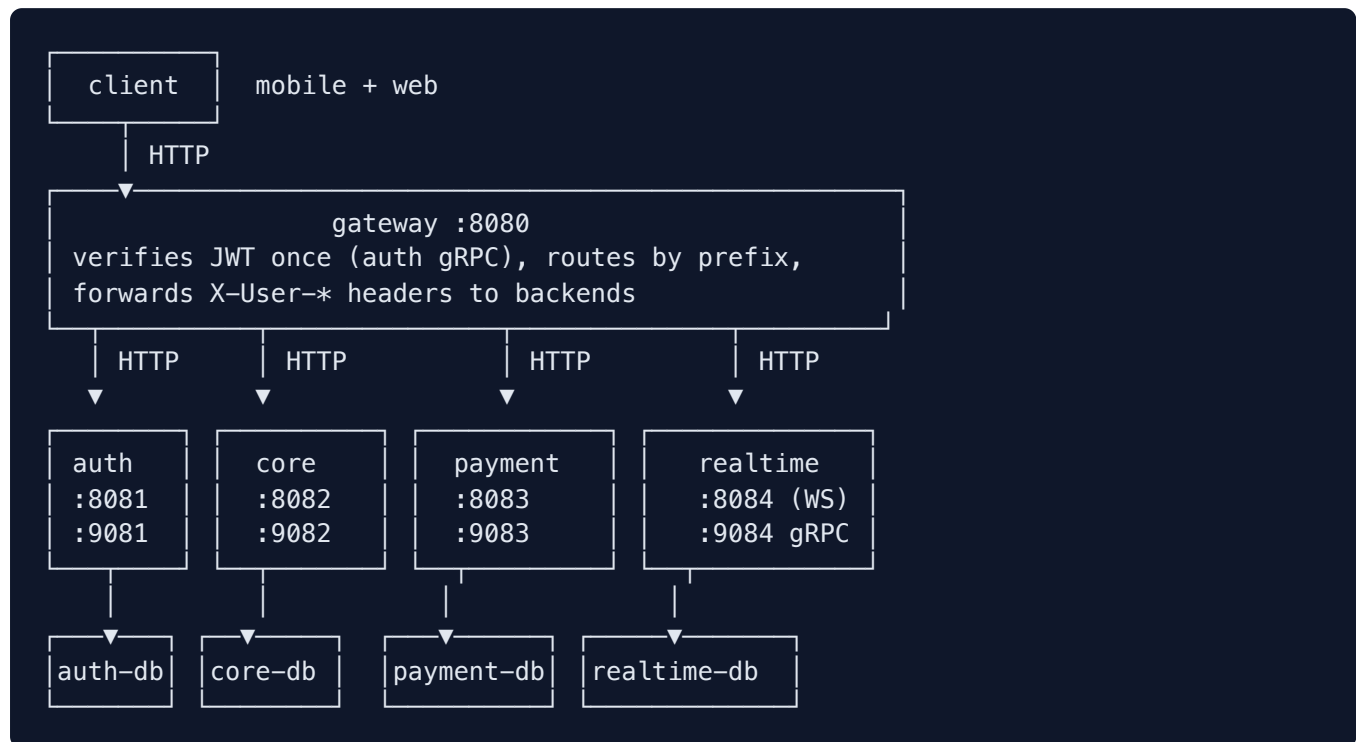
Three Layers, One Repository

Layer	Tech
Backend	Go 1.24, gRPC + Protocol Buffers (internal), HTTP/JSON (public), PostgreSQL 14 per-service, JWT, PayBox
Web	Next.js 16 (App Router, Cache Components, PPR), React 19.2, TypeScript, Tailwind v4, Feature-Sliced Design, Playwright E2E
Mobile	Flutter 3.24 (Impeller engine), Riverpod + MVVM, Clean Architecture, go_router, Dio, Firebase Messaging, secure storage
AI	Anthropic Claude Sonnet 4.6 with tool use; Model Context Protocol (MCP) servers used during development
Ops	Docker Compose, per-service migrations, GitHub Actions CI

Each layer chosen against a measured 2025–2026 industry benchmark — see Slide 13.

Slide 6 — Backend Architecture

Five Go Microservices



- **One DB per service** — strict ownership, no cross-DB joins.
 - **gRPC inter-service** — typed contracts, ~5–10× REST throughput.
 - **HTTP at the edge** — browser-friendly JSON, public OpenAPI.
-

Slide 7 — Service-to-Service Communication

gRPC Contract Map (16 RPCs)

```
core  —————> auth    GetUser, GetUsersBatch, VerifyToken
core  —————> payment   Charge, Refund
core  —————> realtime   EnsureThread, PublishEvent
payment —> core         MarkBookingPaid      (saga callback)
realtime —> auth        GetUsersBatch        (peer-name enrich)
gateway —> auth         VerifyToken          (once per request)
```

Payment saga (synchronous): `core.booking.Pay` → `payment.Charge` → `core.MarkBookingPaid`. If the callback fails, the payment row is still captured; reconciliation is a follow-up job.

Why not async events? Diploma scale; the synchronous saga is simpler to reason about and demonstrate. Slide 14 notes the migration path to async (Kafka or NATS) for production.

Slide 8 — Clean Architecture per Service

Backend Layers (example: `core` service)

```
backend/services/core/internal/  
├─ domain/           Pure Go types: Vendor, Booking, Review (no I/O)  
├─ ports/            Interfaces: VendorRepo, PaymentClient, Notifier  
├─ usecase/          Business logic: vendor/, booking/, review/, photo/  
└─ adapter/  
    ├─ http/         chi router, JSON marshalling, middleware  
    ├─ grpc/         gRPC server impl (core.proto)  
    ├─ grpcclient/   Outbound clients to auth/payment/realtime  
    ├─ repo/         PostgreSQL (sqlx)  
    └─ push/         FCM HTTP v1 sender
```

Layers depend inward only: `adapter → usecase → domain`. **Dependency Inversion** keeps usecase pure and testable — table-driven Go tests stub the ports.

Slide 9 — Web Frontend (Feature-Sliced Design)

Next.js 16 + FSD

```
frontend/src/
├── app/           Routes (App Router) – vendors, bookings, threads, admin
├── features/      ai-chat, auth, booking-flow, messaging, reviews,
                  services, theme, vendor-search
├── entities/      Domain types shared across features
├── widgets/       Composed UI blocks
└── shared/        UI kit, API client, i18n dictionary, hooks
```

Why FSD? Strict horizontal isolation between features prevents the “everything imports everything” decay common in `src/components/` flat layouts. Dependency rule: `shared` → `entities` → `features` → `widgets` → `app` .

Next.js 16 features used: Cache Components with `"use cache"` directive, partial pre-rendering for vendor listings (static shell + streamed availability), React Server Components for vendor detail pages.

Slide 10 — Mobile Architecture

Flutter, Riverpod, MVVM + Clean Architecture

```
mobile/lib/
├── core/          network (Dio + auth interceptor), router, theme, i18n
├── features/      13 feature modules – auth, vendor_catalog, booking,
                  ai_chat, payment, cards, reviews, messaging,
                  notifications, vendor_self, admin, onboarding,
                  settings
├── <feature>/
│   ├── data/      DTOs + datasources + repository impl
│   ├── domain/    Entities + abstract repo + use cases
│   └── presentation/ ViewModels (Riverpod) + screens
└── main.dart
```

- **Riverpod 2.x** for state + DI — compile-safe, no `BuildContext` .
 - **go_router** declarative navigation with deep-link support.
 - **Dio + interceptor** for automatic JWT refresh on 401.
 - **flutter_secure_storage** for refresh-token persistence (Keychain / Keystore).
-

Slide 11 — Data Model

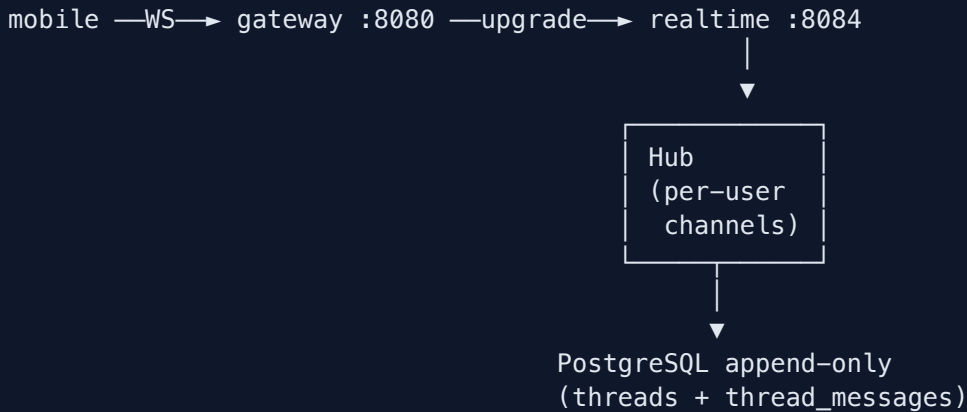
16 Tables across 4 PostgreSQL Databases

Database	Tables	Highlights
auth-db	users , refresh_tokens , password_reset_tokens	role enum (customer/vendor/admin), hashed tokens
core-db	vendors , services , photos , bookings , reviews , notifications , fcm_tokens	full-text search (search_tsv GIN index), rating aggregation, BYTEA photo blobs (5 MB cap)
payment-db	cards , payments	last4 + brand stored, PAN never persisted (PSP scope)
realtime-db	threads , thread_messages	one thread per booking, append-only messages

No cross-service foreign keys. `customer_id` in `core.bookings` is a plain UUID — `auth-svc` owns the actual `users` row. Cross-references resolved by batched gRPC (`auth.GetUsersBatch`).

Slide 12 — Real-time Chat (WebSocket)

Realtime Service Internals



- **One thread per booking** (UNIQUE constraint `threads.booking_id`).
 - **Hub** maintains per-user channel map; on incoming message it fans out to both participants if online and persists immediately.
 - **Auth at WS upgrade:** gateway validates JWT via `auth.VerifyToken` gRPC before allowing upgrade.
 - **Push fallback:** if peer offline, `core.PublishEvent` triggers FCM push with deep-link `/threads/<id>`.
-

Slide 13 — Stack Justification (Comparative)

Why Each Choice — Backed by 2025–2026 Data

Layer	Choice	Main Alternative	Why We Picked Ours	Source
Backend	Go 1.24	Node.js / Java	~2.6× faster CPU-bound; gRPC tooling	Netguru
Inter-svc RPC	gRPC + Protobuf	REST/JSON	5–10× throughput, 70–90% smaller payloads	Markaicode
Web	Next.js 16 + React 19	Remix 3 / SvelteKit	Cache Components + PPR stable in v16	Next.js blog
Web arch	Feature-Sliced Design	Atomic / flat	Hard horizontal isolation	feature-sliced.design
Mobile	Flutter 3.24 (Impeller)	React Native	46% vs 35% cross-platform share; 60–120 fps	Stack Overflow 2024
Mobile state	Riverpod	Bloc / Provider	Compile-safe DI, no <code>BuildContext</code>	docs.riverpod.dev
DB pattern	Postgres per service	Shared DB	Service autonomy; Database-per-Service	microservices.io
AI	Claude API + MCP	OpenAI raw	MCP is the cross-vendor standard (Linux Foundation, 10K+ servers, 97M+ SDK installs)	Anthropic
E2E	Playwright	Cypress	Parallel cross-browser, first-class TS	playwright.dev
Deploy	Docker Compose	Kubernetes	Fits diploma scope; Docker +17pt in 2025	SO 2025 survey

Slide 14 — Comparative Analysis (Competitors)

How Qonaqzhai Differs

Platform	Region	Model	AI	Mobile	KZ Traditions
The Bash	US	Commission per booking	No	iOS+Android	No
GigSalad	US	Freemium subscription	Limited	iOS+Android	No
Peerspace	US	Commission + escrow	Recs	Yes	No
The Knot	US	Listing + ads	Checklist	iOS+Android	No
Wedding.ru	RU	Paid listings	No	Web-first	No
FlyBride	RU	Lead-gen	No	Limited	No
Eventie.kz	KZ	Paid listings	No	Web	Partial
weddingsalon.kz	KZ	Directory	No	Web	Partial
Qonaqzhai	KZ	Commission + escrow (roadmap)	AI planner with tool-use	Native Flutter	Yes (бесбармак, домбра, беташар bundles)

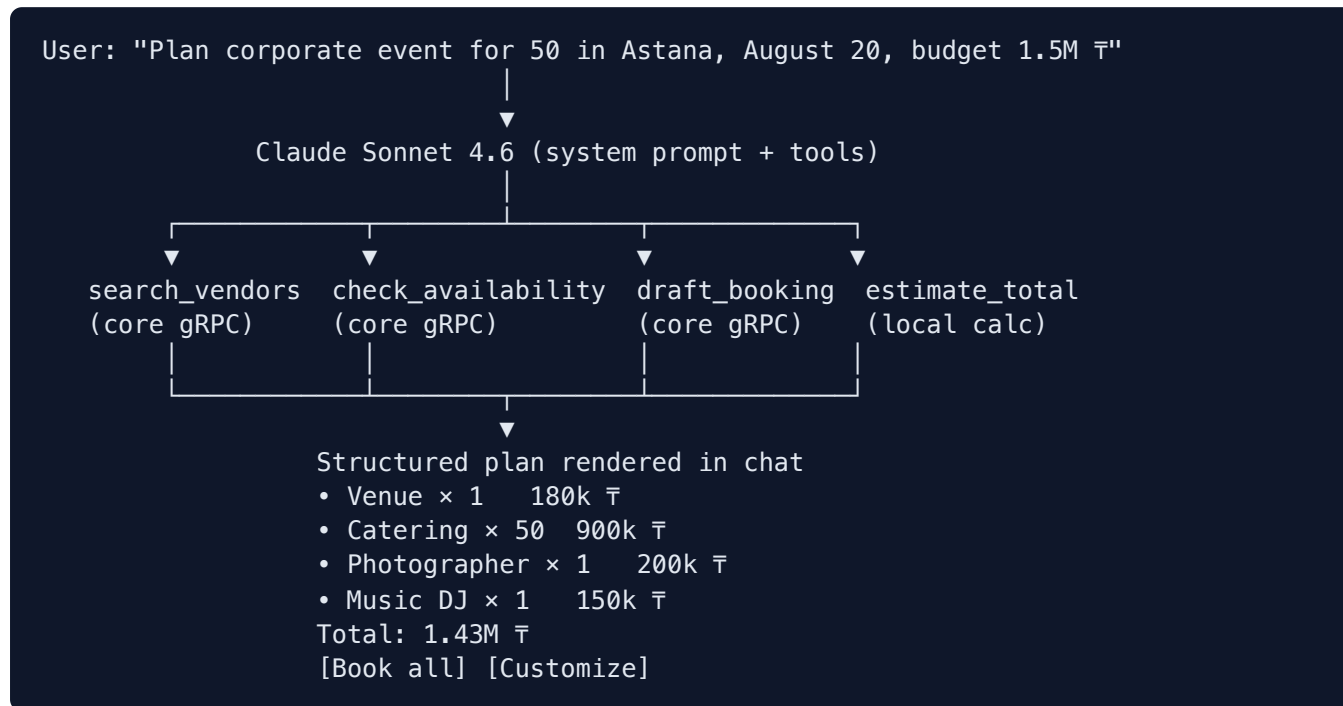
Gaps Qonaqzhai fills:

- 1. **Tri-lingual UX** (KZ / RU / EN) — none of the above support Kazakh first-class.
 - 2. **PayBox integration** — local card schemes, KZT-native.
 - 3. **AI event planner** — Claude-powered, calls backend tools to draft full event plans.
 - 4. **Mobile-first** — competitors are web-led with companion apps; Qonaqzhai is Flutter-native.
 - 5. **Tradition packages** — pre-built bundles for traditional Kazakh ceremonies.
-

Slide 15 — AI Event Planner (Signature Feature)

Claude with Tool Use

The chat is more than Q&A — Claude **calls real backend tools** to build the plan:



Implementation: Anthropic Messages API, `tools` parameter with JSON schemas, streaming responses via SSE from `core` service.

Slide 16 — Security Model

Defense in Depth

Layer	Mechanism
Transport	HTTPS terminated at gateway (production); cert via Let's Encrypt
Authentication	JWT (HS256), 15 min access + 7 day refresh; refresh tokens stored hashed
Authorization	RBAC middleware (<code>customer</code> , <code>vendor</code> , <code>admin</code>); per-route guards
Edge verification	Gateway calls <code>auth.VerifyToken</code> gRPC on every inbound request
Service verification	Each service re-verifies JWT — defense in depth, no implicit trust
Rate limiting	Per-IP 20–40 req/10s on every service
Password reset	One-time short-TTL tokens; email delivery via SMTP
Payments (PCI)	Card PAN never persisted in our DB; PayBox PSP holds the scope
Photo uploads	MIME whitelist (JPEG/PNG/WebP/GIF), 5 MB cap, content-type sniffed
Cross-DB joins	Forbidden — no shared schema, no leaked FK references

Slide 17 — Testing & Quality

What We Test

Component	Files	LOC	Approach
Auth service	1	386	Signup, login, refresh, password reset
Core service	4	1,007	Vendor CRUD, booking state machine, review aggregation
Payment service	2	436	Card validation, charge / refund, PSP mock
Realtime service	1	204	Thread ensure, message send, WS connection
Backend total	8	2,033	Table-driven Go tests, <code>-race</code> enabled
Frontend E2E	12 specs	~800	Playwright — auth, booking, chat, admin, vendor flows
Mobile	ViewModel tests	growing	Riverpod <code>ProviderContainer</code> mocks

CI gate: backend tests + Playwright E2E must pass before merge.

Slide 18 — Agentic Engineering & MCP

How This Diploma Was Built

The diploma was built using **agentic engineering** workflows — Claude Code with multiple MCP servers acted as the second engineer.

MCP servers used in development:

- **GitHub MCP** — branch / PR / issue automation
- **Filesystem MCP** — repo-wide refactors with safety checks
- **Postgres MCP** — schema introspection during migration writing
- **Context7 MCP** — pulled latest Next.js 16 + Riverpod docs (project pinned to Jan 2026 cut-off otherwise)
- **Exa / WebSearch** — competitor research, benchmark data for Slide 13

Why this matters for 2026: Anthropic donated MCP to the **Linux Foundation's Agentic AI Foundation** in 2025; 10K+ MCP servers are live, 97M+ SDK installs. Stack Overflow 2025 reports 84% of developers use AI coding tools. This diploma reflects how engineering is actually done today.

Slide 19 — Live Demo Script (8 min)

Defense Demo Flow

1. **Localization** (30s) — Open mobile, switch KZ → RU → EN.
 2. **AI Planner** (2 min) — "Plan corporate event for 50 in Astana, 20 Aug, 1.5M ₸" → Claude returns 4-vendor plan with reasoning.
 3. **Booking + Calendar** (1 min) — Tap venue, view availability, confirm Aug 20.
 4. **Payment** (1 min) — PayBox test card → push notification fires → tap → deep-link into booking.
 5. **Vendor side** (1 min) — Log in as vendor, see new booking in analytics dashboard, revenue chart updates.
 6. **Admin moderation** (30s) — Web admin moderates a flagged review.
 7. **Architecture trace** (1 min) — Show `docker compose logs` of saga: `core → payment.Charge → core.MarkBookingPaid`.
 8. **Q&A** (1 min).
-

Slide 20 — Roadmap

What Ships Before Defense vs. Future Work

Tier 1 — Shipped / in flight before defense

- AI event planner with tool use
- Vendor analytics dashboard
- KZ / RU / EN i18n completion
- Push notifications with deep-links
- Chat polish (typing indicators, read receipts)

Tier 2 — Next iteration (post-diploma)

- Smart vendor recommendations (embedding-based)
- Dynamic pricing (peak / off-peak)
- Google Calendar sync for vendor availability
- Referral / promo codes
- Verified-booking review badge

Tier 3 — Future research

- Escrow with milestone release
 - Voice input for AI planner
 - ML matching via sentence embeddings
 - Vendor on-boarding KYC via egov.kz
 - AI-generated minute-by-minute event runbook
-

Slide 21 — Lessons Learned

Honest Retrospective

What Worked	What I Would Change
One DB per service — clear ownership, no schema fights	Synchronous payment saga is fragile; should publish events (NATS / Kafka)
Riverpod for Flutter — fewer rebuilds, easy testing	Started with hand-rolled DTOs; should generate from OpenAPI from day 1
FSD on the web — features stay isolated	Tailwind v4 + Next.js 16 still rough edges; some hydration warnings remain
Claude + MCP during dev — 2–3× productivity on boilerplate	Need stricter prompt discipline; early sessions drifted scope
Per-service Go modules — independent build & test	Workspace <code>replace</code> directives are subtle; broke CI twice
gRPC for internal — no API drift	Browser debugging is painful; we keep REST at the gateway for a reason

Slide 22 — Q&A

Thank you.

Repo: `github.com/Bahaidahar/diploma` **Live demo:** see Slide 19 **Architecture deep-dive:** see `presentation/research/code-map.md` **Comparative analysis:** see `presentation/research/competitors.md` **Stack justification:** see `presentation/research/stack-justification.md`

Appendix A — Numbers at a Glance

- **5** Go microservices
 - **4** PostgreSQL databases
 - **16** gRPC methods
 - **68** public HTTP endpoints
 - **16** database tables
 - **13** Flutter feature modules
 - **16** Next.js routes
 - **8** Go test files, **2,033** LOC
 - **12** Playwright E2E specs
 - **3** languages supported (RU / KZ / EN)
 - **1** integrated Claude AI planner with tool use
-

Appendix B — How to Read This Repository

```
diploma/
├── README.md
├── backend/
│   ├── proto/
│   │   └── .proto source – contract of the system
│   ├── gen/proto/
│   │   └── generated Go gRPC stubs
│   ├── pkg/
│   │   └── shared: auth, errs, httpx, grpcutil
│   ├── services/
│   │   └── auth, core, payment, realtime, gateway
│   └── tests/e2e/
│       └── Docker-backed end-to-end suite
├── frontend/
│   ├── src/app/
│   │   └── routes
│   ├── src/features/
│   │   └── feature slices
│   └── e2e/
│       └── Playwright specs
├── mobile/
│   └── lib/features/
│       └── 13 feature modules
├── presentation/
│   ├── diploma.md
│   │   └── this defense package
│   ├── slides (this file)
│   └── research/
│       ├── competitors.md
│       ├── stack-justification.md
│       ├── code-map.md
│       └── features-roadmap.md
```

Start reading from `backend/proto/` — the contracts define the system.